

Project Title

AI-Based Text Summarization Platform

Objective

The goal of this project is to design and develop a web-based application that can:

- Accept user input (text or PDF files)
- Process the content using AI models
- Generate a concise and meaningful summary
- Display results in a structured format

Students will gain hands-on experience in:

- Full-stack web development
 - API integration
 - Working with AI/ML models
 - Handling real-world data
-

Project Overview

In today's world, large volumes of textual data are generated daily. This project aims to build a system that automatically summarizes long content into shorter, meaningful insights using Natural Language Processing (NLP).

The system follows this workflow:

User Input → Backend Processing → AI Model → Summary Output

System Architecture

The application consists of three main components:

1. Frontend

- Provides user interface
- Accepts input (text/PDF)
- Displays summarized output

2. Backend

- Handles API requests
- Processes uploaded files
- Communicates with AI model

3. AI Engine

- Performs text summarization
 - Uses pre-trained NLP models or APIs
-

Technology Stack

Students can choose from the following (recommended stack provided):

Frontend

- Next.js / React.js
- HTML, CSS, Tailwind CSS

Backend

- Node.js with Express
- OR**
- Python FastAPI

AI Integration

- OpenAI API (recommended for simplicity)
- OR**
- HuggingFace Transformers

Additional Libraries

- pdf-parse (PDF text extraction)
 - axios / fetch (API calls)
-

Functional Requirements

The system must include the following features:

1. Input Handling

- Accept text input from user
- Allow PDF file upload

2. Text Processing

- Extract text from uploaded files
- Clean and preprocess text

3. Summarization

- Generate summary using AI
- Allow control over summary length (optional)

4. Output Display

- Show summary clearly
 - Format output (paragraphs or bullet points)
-

Implementation Steps

Step 1: Project Setup

- Create a frontend application using React or Next.js
 - Initialize backend server (Node.js or FastAPI)
-

Step 2: Build User Interface

Design a simple UI with:

- File upload option
 - Text input box
 - “Summarize” button
 - Output display section
-

Step 3: File Handling

- Implement file upload functionality
 - Extract text from PDF using libraries
-

Step 4: Backend API Development

Create an API endpoint:

- Accepts text input
 - Sends data to AI model
 - Returns summarized output
-

Step 5: AI Integration

Students can choose one:

Option A: API-Based Approach

- Use OpenAI API for summarization

Option B: Model-Based Approach

- Use HuggingFace summarization models
-

Step 6: Connect Frontend and Backend

- Send user input from frontend to backend
 - Display returned summary
-

Step 7: Testing

Test the system with:

- Short text
 - Long articles
 - PDF documents
-

Datasets (Optional for Advanced Students)

Students who want to train their own models can use:

- CNN/DailyMail Dataset
- XSum Dataset
- BillSum Dataset
- ArXiv Dataset

These datasets are available on HuggingFace and contain text-summary pairs.

Optional Advanced Features

Students can implement additional features for higher grades:

- Multi-language summarization
 - Keyword extraction
 - Chat with document
 - Export summary as PDF
 - User authentication system
-

Evaluation Criteria

Criteria	Marks
Functionality	30
UI/UX Design	15
Code Quality	15
AI Integration	20
Innovation	10
Documentation	10

Expected Output

The final system should:

- Accept input (text/PDF)
 - Generate a clear summary
 - Display output in readable format
-

Guidelines

- Use modular coding practices
 - Maintain proper folder structure
 - Handle errors (invalid input, file issues)
 - Ensure clean UI design
 - Avoid plagiarism
-

Deliverables

Students must submit:

1. Source Code
 2. Project Report (PDF) (optional but recommended)
 3. Demo Video
 4. README file with setup instructions
-

Learning Outcomes

By completing this project, students will:

- Understand NLP basics
 - Learn API integration
 - Build full-stack applications
 - Work with real-world AI systems
-

Students are encouraged to:

- Start simple (text summarization first)
- Then extend to PDF and advanced features
- Focus on understanding rather than copying code